

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2
по дисциплине «Операционные системы»
Вариант «FIFO»

Выполнил:
Печкуров Данила Алексеевич
Группа: Р3308
Преподаватель:
Тюрин Иван Николаевич

г. Санкт-Петербург
2024-5 г.

Задание

Для оптимизации работы с блочными устройствами в ОС существует кэш страниц с данными, которыми мы производим операции чтения и записи на диск. Такой кэш позволяет избежать высоких задержек при повторном доступе к данным, так как операция будет выполнена с данными в RAM, а не на диске (вспомним пирамиду памяти).

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки (dll или so). Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

При выполнении работы необходимо реализовать простой API для работы с файлами, предоставляющий пользователю следующие возможности:

1. Открытие файла по заданному пути файла, доступного для чтения. Процедура возвращает некоторый хэндл на файл. Пример:
`int lab2_open(const char *path).`
2. Закрытие файла по хэндлу. Пример:
`int lab2_close(int fd).`
3. Чтение данных из файла. Пример:
`ssize_t lab2_read(int fd, void buf[.count], size_t count).`
4. Запись данных в файл. Пример:
`ssize_t lab2_write(int fd, const void buf[.count], size_t count).`
5. Перестановка позиции указателя на данные файла. Достаточно поддерживать только абсолютные координаты. Пример:
`off_t lab2_lseek(int fd, off_t offset, int whence).`
6. Синхронизация данных из кэша с диском. Пример:
`int lab2_fsync(int fd).`

Операции с диском разработанного блочного кэша должны производиться в обход page cache используемой ОС.

В рамках проверки работоспособности разработанного блочного кэша необходимо адаптировать указанную преподавателем программу-загрузчик из ЛР 1, добавив использование кэша. Запустите программу и убедитесь, что она корректно работает. Сравните производительность до и после.

Ограничения

- Программа (комплекс программ) должна быть реализован на языке C или C++.

- Если по выданному варианту задана политика вытеснения Optimal, то необходимо предоставить пользователю возможность подсказать page cache, когда будет совершен следующий доступ к данным. Это можно сделать либо добавив параметр в процедуры read и write (например, ssize_t lab2_read(int fd, void buf[.count], size_t count, access_hint_t hint)), либо добавив еще одну функцию в API (например, int lab2_advice(int fd, off_t offset, access_hint_t hint)). access_hint_t в данном случае – это абсолютное время или временной интервал, по которому разработанное API будет определять время последующего доступа к данным.
- Запрещено использовать высокоуровневые абстракции над системными вызовами. Необходимо использовать, в случае Unix, процедуры libc.

Выполнение

Реализованы блочный кэш (с политикой вытеснения FIFO) и API для работы с файлами, который использует блочный кэш в обход системного
Код:

В **CacheManager.h** определен класс **CacheManager**, который управляет кэшированием блоков данных, а также содержит структуру **Block**, представляющую блок данных с указателем на массив и размером. Этот класс реализует основные операции с кэшем, такие как чтение, запись, синхронизация данных с диском и вытеснение наименее часто используемых блоков (FIFO).

В **CacheManager.cpp** реализованы методы для чтения и записи блоков данных в кэш, синхронизации кэша с диском, а также вытеснения наименее часто используемых блоков, если кэш заполнен. Методы взаимодействуют с файловой системой через Windows API для чтения и записи данных.

В **CacheAPI.h** объявлены функции API для работы с файлами через кэш, включая открытие, закрытие, чтение, запись, перемещение указателя в файле и синхронизацию данных с диском, которые обеспечивают доступ к файлам с учетом кэширования.

В **CacheAPI.cpp** реализованы функции API, использующие методы **CacheManager** для выполнения операций с кэшем при чтении и записи данных в файлы.

В **app.cpp** находится основная программа, которая демонстрирует работу с различными режимами чтения файлов: с кэшированием, без кэширования и с использованием FIFO-кэширования. Программа создает файл размером 1 ГБ, выполняет несколько случайных операций чтения и измеряет время выполнения для каждого подхода.

Результаты выполнения

```
Running without cache:  
Reading without cache took 2.1588 ms  
Running with cache:  
Reading with cache took 0.0679 ms  
Running with FIFO cache:  
Reading with FIFO cache took 2.213 ms
```

```
Running without cache:  
Reading without cache took 1.9674 ms  
Running with cache:  
Reading with cache took 0.0721 ms  
Running with FIFO cache:  
Reading with FIFO cache took 93.8784 ms
```

```
Running without cache:  
Reading without cache took 2.1321 ms  
Running with cache:  
Reading with cache took 17.2711 ms  
Running with FIFO cache:  
Reading with FIFO cache took 1.907 ms
```

```
Running without cache:  
Reading without cache took 3.2579 ms  
Running with cache:  
Reading with cache took 0.0954 ms  
Running with FIFO cache:  
Reading with FIFO cache took 2.23 ms
```

Вывод

В ходе работы был реализован блочный кэш в пространстве пользователя с политикой FIFO, работающий в обход системного. Тестирование с программой-нагрузчиком

показало, что кэширование значительно ускоряет доступ к данным (до 0.0679 мс против 1.96–3.25 мс без кэша). Однако FIFO-кэш продемонстрировал нестабильную производительность, особенно при случайном доступе к блокам, когда старые страницы вытеснялись до их повторного использования. В одном из тестов это привело к резкому увеличению времени чтения (93.88 мс). Для повышения эффективности в таких сценариях целесообразно использовать более адаптивные алгоритмы, например, LRU. В целом, кэш работает корректно, но его производительность зависит от характера нагрузки.